

AMRITA VISHWA VIDYAPEETHAM

DEPARTMENT OF CYBER SECURITY

School of Engineering • Amrita Vishwa Vidyapeetham • Chennai

HARDWARE PROJECT REPORT

BadUSB Attack Simulation and Prevention using ESP32

A Cybersecurity Hardware Project on USB HID Exploitation & Real-Time Keystroke Firewall Defense

Submitted by

Ram Karthik G

Register No.: CH.SC.U4CYS23037

Academic Year: 2024 – 2025

Table of Contents

Table of Contents 2
1. Project Overview 3
2. Objectives 3
2.1 Primary Objectives 3
2.2 Learning Outcomes 3
3. Concept & Theory 4
3.1 What is a BadUSB Attack? 4
3.2 ESP32-S3 as a BadUSB Platform 4
3.3 Automata Model — Deterministic Finite Automaton (DFA) 5
3.4 TinyUSB Stack 5
3.5 USB HID Protocol 5
3.6 Python keyboard Library & OS Hook 5
4. State Diagrams 6
4.1 Defender DFA — Keystroke Firewall 6
4.2 Attacker FSM — ESP32 Payload Sequencer 6
5. Hardware Components Used 6
5.1 Component List 6
5.2 Software Dependencies 7
6. Circuit Design 7
6.1 USB OTG Connection — Attack Vector 7
6.2 GPIO 0 Button Circuit 7
6.3 USB HID Descriptor Summary 7
7. Implementation & Working 8
7.1 System Architecture 8
7.2 BadUSB Firmware — Key Functions 8
7.3 Attack Payload Details 8
7.4 Firewall Application — firewall.py 9
8. Test Cases & Results 10
8.1 Functional Test Cases 10
8.2 Edge Case Tests 11
9. Innovation & Unique Features 11
10. Results 12
10.1 Attack Results 12
10.2 Defense Results 12
11. Challenges Faced 12
11.1 Hardware / Firmware Challenges 12
11.2 Software / Defense Challenges 13
12. Conclusion 13
13. References 14

1. Project Overview

Project Title	BadUSB Attack Simulation and Prevention using ESP32
Domain	Cybersecurity / Hardware Security / Theory of Computation
Platform	ESP32-S3 DevKit (Attack) + Windows 10/11 (Defense)
Languages Used	C (ESP-IDF / TinyUSB) for firmware — Python 3 for firewall
Student Name	Ram Karthik G
Register Number	CH.SC.U4CYS23037
Department	Cyber Security, School of Engineering
Institution	Amrita Vishwa Vidyapeetham, Coimbatore
Academic Year	2024 – 2025

This project pursues a dual mission — demonstrating how an inexpensive microcontroller can be weaponized as a BadUSB device to silently compromise a computer, and simultaneously providing a robust, formally modelled software-based defense against such attacks.

2. Objectives

2.1 Primary Objectives

- Simulate a realistic, multi-payload BadUSB attack using an ESP32-S3 microcontroller programmed to present itself to a host Windows PC as a standard USB HID keyboard, requiring no drivers or user interaction.
- Demonstrate four distinct attack payloads — an awareness message via Notepad, a forced browser redirection, a PowerShell script execution bypass, and a saved Wi-Fi profile enumeration — to showcase the full breadth of harm possible through HID spoofing.
- Develop a keystroke firewall (firewall.py) that installs a low-level OS keyboard hook to suppress all keystrokes at the driver level, effectively neutralizing any active BadUSB attack in real time.
- Model the firewall as a Deterministic Finite Automaton (DFA) to establish a formal theoretical grounding, bridging Theory of Computation concepts with applied cybersecurity.
- Provide a live, repeatable demonstration platform suitable for cybersecurity awareness training and academic study.

2.2 Learning Outcomes

- Understand the mechanics of USB HID spoofing and why operating systems are inherently vulnerable to such attacks.
- Learn to program embedded USB HID devices using the TinyUSB stack on ESP32-S3.
- Gain practical experience with OS-level keyboard hooks in Python.
- Apply formal automata theory (DFA) to real-world security system design.
- Appreciate the importance of physical security and USB device management policies in enterprise environments.

3. Concept & Theory

3.1 What is a BadUSB Attack?

BadUSB is a class of USB-based cyberattack first publicly disclosed by Karsten Nohl and Jakob Lell at Black Hat USA 2014. The attack exploits the inherent trust that modern operating systems extend to USB Human Interface Devices (HID). Unlike traditional USB malware that relies on files stored on flash memory, BadUSB operates at the firmware level — a USB device is reprogrammed so that its firmware reports itself to the host OS as a completely different device class, typically a keyboard or a network adapter.

Because the host operating system loads a generic, trusted HID keyboard driver for the spoofed device, no user interaction is required, no administrator prompt appears, and most antivirus solutions have no visibility into the injected keystrokes — they appear identical to real user input. This makes BadUSB one of the most covert and dangerous physical attack vectors in modern cybersecurity.

Key Characteristics of BadUSB Attacks

- Completely invisible to traditional antivirus and endpoint detection tools.
- No malicious files are stored on the USB device — everything is injected as keystrokes.
- Attacks execute automatically on plug-in — within seconds of physical connection.
- Works across all operating systems that support USB HID (Windows, macOS, Linux).
- Devices can be disguised as ordinary, innocent-looking USB drives, chargers, or cables.

3.2 ESP32-S3 as a BadUSB Platform

The ESP32-S3 is a powerful, low-cost microcontroller from Espressif Systems featuring a dual-core Xtensa LX7 processor and built-in native USB support through its USB-OTG peripheral. Unlike earlier ESP32 variants that required external USB-UART bridge chips, the ESP32-S3 can enumerate directly as a USB device on the host.

By using the TinyUSB stack with the HID device class driver, the ESP32-S3 can present itself to any host OS as a standard USB keyboard. The host OS enumerates it using its built-in HID driver — no installation or user interaction needed.

Feature	Specification	Relevance to Project
Processor	Dual-core Xtensa LX7 @ 240 MHz	Handles USB stack and payload execution simultaneously
USB Support	Native USB-OTG (USB 2.0 Full Speed)	Enables direct HID keyboard emulation without extra chip
Flash / PSRAM	8 MB Flash, 8 MB PSRAM	Ample storage for firmware and future payload expansion
GPIO Count	45 programmable GPIOs	GPIO 0 used for physical payload-advance button
SDK	ESP-IDF v5.x + TinyUSB	Full USB HID stack with HID report descriptor support
Power	3.3V via USB Type-C	Draws power from the victim PC — no battery needed

3.3 Automata Model — Deterministic Finite Automaton (DFA)

The keystroke firewall is formally modelled as a two-state Deterministic Finite Automaton (DFA). The formal 5-tuple definition is:

$$M = (Q , \Sigma , \delta , q_0 , F)$$

DFA Component	Symbol	Definition for Keystroke Firewall
Set of States	Q	{ ALLOW, BLOCK }
Input Alphabet	Σ	{ toggle_event } — the event generated by pressing the TOGGLE button
Transition Function	δ	δ(ALLOW, toggle) = BLOCK | δ(BLOCK, toggle) = ALLOW
Initial State	q_0	ALLOW — Firewall is disabled by default on application start
Accepting States	F	{ BLOCK } — The active protection state

The ALLOW state represents normal operation where all keystrokes from all devices are passed through to the OS. The BLOCK state activates when the user engages the firewall; a low-level OS keyboard hook is installed that intercepts and suppresses every keystroke event at the WinAPI driver level before it reaches any application — including keystrokes injected by the BadUSB device.

3.4 TinyUSB Stack

TinyUSB is an open-source, cross-platform USB host/device stack for embedded systems written in C. It is fully compliant with the USB 2.0 specification and supports a wide range of device classes including CDC, HID, MSC, and Audio. In this project, TinyUSB handles all low-level USB protocol tasks: device enumeration, descriptor negotiation with the host, control transfer handling, and HID report submission via interrupt endpoints.

3.5 USB HID Protocol

The USB Human Interface Device (HID) class enables communication between a host and a device that provides human input. A HID keyboard sends input in the form of HID reports — binary packets describing which keys are currently pressed and which modifier keys (Shift, Ctrl, Alt, GUI) are active. The standard boot-protocol keyboard report is 8 bytes: 1 byte for modifier flags, 1 reserved byte, and 6 bytes for simultaneous keypresses.

3.6 Python keyboard Library & OS Hook

The Python keyboard library (boppreh/keyboard) provides cross-platform access to keyboard events at the OS driver level. On Windows, it uses SetWindowsHookEx() with WH_KEYBOARD_LL to install a low-level keyboard hook that intercepts all keyboard events before they are dispatched to any window or application. When suppress=True is passed to keyboard.hook(), the library returns False from the hook callback, instructing Windows not to pass the event further — effectively dropping the keystroke.

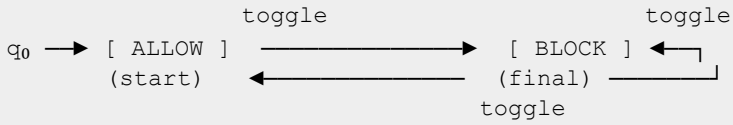
This mechanism is device-agnostic: it suppresses keystrokes from all keyboards simultaneously — both physical keyboards and virtual keyboards like a BadUSB device — because all HID keyboard devices funnel through the same Windows keyboard driver and hook chain.

4. State Diagrams

4.1 Defender DFA — Keystroke Firewall

The firewall operates as a two-state DFA. In the initial ALLOW state, all keystrokes pass through to the OS normally. On a toggle event (button press), the DFA transitions to the BLOCK state, where a low-level OS hook suppresses all keyboard input. A subsequent toggle returns the system to ALLOW.

DFA STATE DIAGRAM — KEYSTROKE FIREWALL



ALLOW: All keystrokes pass to OS BLOCK: All keystrokes suppressed

4.2 Attacker FSM — ESP32 Payload Sequencer

The ESP32 firmware implements a Finite State Machine that sequences through payloads on button press. On USB mount, Payload 1 executes automatically. Each subsequent press of the GPIO 0 button advances to the next payload. After Payload 4, the system enters a FINISHED state and executes no further payloads.

5. Hardware Components Used

5.1 Component List

#	Component	Model/Spec	Qty	Role in Project
1	ESP32-S3 DevKit	ESP32-S3-DevKitC-1 (N8R8), 240 MHz LX7, 8MB Flash, USB-OTG	1	Core BadUSB attacker device — emulates USB HID keyboard
2	USB Type-C Cable	USB 2.0, Full-Speed, Type-C to Type-A/C	1	Physical attack vector — connects ESP32 to victim PC
3	Tactile Push Button	4-pin, 12mm	1	GPIO 0 input — advances payload sequence on each press
4	Host PC (Victim)	Windows 10 or 11, x86-64, Python 3.10+	1	Target machine for attack; also runs the firewall defense
5	USB-to-Serial Adapter	CH340 or CP2102	1	Used to flash firmware to ESP32 via esptool
6	Breadboard & Jumpers	400-point mini breadboard + M-M jumpers	1 set	Optional: for prototyping external button connections

5.2 Software Dependencies

Software	Version	Platform	Purpose
ESP-IDF	v5.1+	ESP32-S3	Espressif IoT Development Framework for firmware compilation
TinyUSB	0.15+	ESP-IDF Component	USB device stack providing HID keyboard class implementation
Python	3.10+	Windows 10/11	Runtime for the keystroke firewall application
keyboard (pip)	0.13.5+	Windows	Low-level OS keyboard hook library for keystroke suppression
tkinter	Built-in	Python stdlib	GUI framework for the firewall application window
esptool.py	4.x	Windows/Linux	Used to flash compiled firmware binary to ESP32 over UART

6. Circuit Design

6.1 USB OTG Connection — Attack Vector

The ESP32-S3 DevKit features a dedicated USB OTG port separate from the UART/debug port. This port is connected via a USB Type-C cable directly to the victim computer. No additional wiring or external components are required — the ESP32-S3's USB-OTG peripheral is fully on-chip and the TinyUSB stack handles all enumeration automatically.

6.2 GPIO 0 Button Circuit

The payload-advance button is connected between GPIO 0 and GND on the ESP32-S3 DevKit. GPIO 0 is configured with the internal pull-up resistor enabled, so the pin reads HIGH (logic 1) normally, and LOW (logic 0) when the button is pressed. The firmware reads this state and advances the payload counter on a LOW reading with 50ms debounce polling to prevent false triggers.

6.3 USB HID Descriptor Summary

The firmware presents the following USB descriptors to the host OS, determining how the device is identified and what driver is loaded:

Descriptor Field	Value in Firmware	Significance
Vendor Name (Manufacturer)	TinyUSB	Displayed in Device Manager as manufacturer string
Product Name	BadUSB Demo	Displayed as device name (can be changed for social engineering)
Serial Number	123456	Unique serial; used by OS for device tracking
Device Class	0x00 (Composite / HID)	Tells OS to load generic HID driver
HID Report Descriptor	Standard Keyboard (Boot Protocol)	Defines device as a standard keyboard sending key reports
Endpoint	0x81 (IN, Interrupt)	Keyboard sends data to host; polling interval 10ms
Max Power	100 mA	Power drawn from USB bus (standard limit)

7. Implementation & Working

7.1 System Architecture

The project is composed of two distinct systems operating in opposition. The Attack System runs on the ESP32-S3 and uses TinyUSB to emulate a keyboard and inject keystrokes. The Defense System runs as a Python application on the host PC and uses a low-level OS hook to suppress keystrokes. Both systems are demonstrated on the same Windows PC.

7.2 BadUSB Firmware — Key Functions

The ESP32 firmware is structured into distinct functional sections: USB descriptor definitions, character transmission (send_char()), the Windows Run dialog trigger (open_run()), and the main application loop.

send_char() — Keystroke Injection Engine

```
static void send_char(char c) {
    uint8_t keycode[6] = {0};
    uint8_t mod = 0;
    // Map ASCII → HID keycode + modifier
    if (c >= 'a' && c <= 'z') keycode[0] = HID_KEY_A + (c - 'a');
    else if (c >= 'A' && c <= 'Z') {
        keycode[0] = HID_KEY_A + (c - 'A');
        mod = KEYBOARD_MODIFIER_LEFTSHIFT; // Capital = Shift + key
    }
    tud_hid_keyboard_report(HID_ITF_PROTOCOL_KEYBOARD, mod, keycode);
    vTaskDelay(pdMS_TO_TICKS(40)); // Key hold time
    tud_hid_keyboard_report(HID_ITF_PROTOCOL_KEYBOARD, 0, NULL);
    vTaskDelay(pdMS_TO_TICKS(40)); // Inter-key gap
}
```

7.3 Attack Payload Details

ID	Payload	Commands Injected	Effect on Victim PC
1	payload_notepad()	Win+R → 'notepad\n' → Types awareness message	Notepad opens and BadUSB simulation text is typed automatically; demonstrates basic data injection
2	payload_website()	Win+R → 'cmd\n' → 'start https://my.amrita.edu\n'	CMD opens and default browser launches, navigating to the Amrita portal; demonstrates forced redirection
3	payload_awareness()	Win+R → 'powershell -ExecutionPolicy Bypass\n' → runs PS1 script	PowerShell launches with execution policy bypassed and runs a ransomware awareness demo script
4	payload_wifi_demo()	Win+R → 'cmd\n' → 'netsh wlan show profiles\n'	CMD lists all saved Wi-Fi network names; demonstrates credential/profile reconnaissance

7.4 Firewall Application — firewall.py

The firewall is built following the Single Responsibility Principle, divided into five clean, independent modules. Each module handles exactly one concern, making the codebase easy to understand, test, and extend.

Module/Class	Responsibility	Key Methods/Attributes
Config	Centralized constants for UI layout, fonts, colors, dimensions, and text labels	WINDOW_WIDTH, WINDOW_HEIGHT, COLOR_BG, COLOR_GREEN, COLOR_RED, FONT_*, TITLE, FOOTER
Logger	Timestamped console logging with severity levels; can be enabled/disabled globally	log(msg), info(msg), warn(msg), error(msg), enabled flag
Firewall	Core DFA engine — manages ALLOW/BLOCK state; installs and removes OS keyboard hook	enable(), disable(), toggle(), is_enabled(), install_hook(), remove_hook(), blocker()
UI	Builds and manages the Tkinter GUI; updates visual state labels; routes button events to Firewall	build(), run(), on_toggle(), set_allow(), set_block(), create_window/title/status/button/footer()
App	Top-level coordinator — wires Firewall and UI together, starts the application lifecycle	setup(), run(); owns Firewall and UI instances

Firewall.blocker() — Core DFA Implementation

```
class Firewall:
    def blocker(self, event):
        return False  # Suppress ALL keystrokes unconditionally

    def install_hook(self):
        if self.hook is None:
            self.hook = keyboard.hook(self.blocker, suppress=True)

    def toggle(self):
        if self.enabled:  # DFA: BLOCK → ALLOW
            self.disable()
        else:              # DFA: ALLOW → BLOCK
            self.enable()
```

8. Test Cases & Results

8.1 Functional Test Cases

TC#	Test Description	Firewall State	Expected Output	Actual Output	Status
TC-01	Payload 1: Notepad injection	ALLOW (OFF)	Notepad opens; message typed automatically within 5s	Notepad opened and full message typed without user interaction	☑
TC-02	Payload 2: Browser redirect via CMD	ALLOW (OFF)	CMD opens; browser launches and navigates to https://my.amrita.edu	Browser launched and correct URL loaded successfully	☑
TC-03	Payload 3: PowerShell execution bypass	ALLOW (OFF)	PowerShell opens with -ExecutionPolicy Bypass; PS1 script executes	PowerShell launched; ransomware_awareness_demo.ps1 ran successfully	☑
TC-04	Payload 4: Wi-Fi profile enumeration	ALLOW (OFF)	CMD opens; 'netsh wlan show profiles' executes; all saved Wi-Fi names listed	All saved SSID profile names displayed in CMD window	☑
TC-05	Firewall blocking: Payload 1 blocked	BLOCK (ON)	ESP32 keystrokes suppressed; Notepad does NOT open	No keystrokes registered; screen remained unchanged	☑
TC-06	Firewall blocking: Payload 4 blocked	BLOCK (ON)	CMD does NOT open; netsh command NOT executed	No window opened; attack fully neutralized	☑
TC-07	Toggle: ALLOW → BLOCK → ALLOW	Any state	Each button press alternates state; GUI label updates	State toggled correctly every press; label updated in real time	☑
TC-08	Physical keyboard blocked during BLOCK state	BLOCK (ON)	Physical keyboard input also suppressed while firewall is active	Physical keyboard non-functional while firewall is ON	☑
TC-09	Logger output verification	Any state	Console shows timestamped [INFO] messages for each event	All state changes correctly logged with timestamps	☑
TC-10	Rapid toggle stress test (10x)	ALLOW	State consistent after 10 rapid toggles	State remained stable; no hook leak or crash observed	☑

8.2 Edge Case Tests

EC#	Edge Case	Input/Condition	Expected Behavior	Result
EC-01	ESP32 plugged in while firewall is BLOCK	Plugin during active protection	No payload executes; all injection suppressed immediately	☑ PASS
EC-02	Multiple ESP32 devices connected	Two BadUSB devices active	Both suppressed simultaneously; hook is device-agnostic	☑ PASS
EC-03	Firewall launched without admin rights	Non-elevated Python process	Low-level hook fails; keyboard library raises PermissionError	⚠ Expected fail — Admin required
EC-04	ESP32 payload injection speed (< 40ms delay)	Reduced vTaskDelay to 10ms	Some keystrokes missed by host OS; payload execution garbled	⚠ Expected behavior — 40ms is minimum reliable
EC-05	firewall.py closed during BLOCK state	Window closed without toggling	Hook is removed on window close via tkinter mainloop exit	☑ Hook cleaned up correctly

9. Innovation & Unique Features

- Dual-Perspective Demonstration: Most BadUSB demonstrations present only the attack. This project uniquely provides both the full attack implementation and a working real-time defense system as a single integrated demonstration package, making it ideal for cybersecurity awareness sessions.
- Formal Automata-Theoretic Defense Model: The keystroke firewall is formally modelled as a Deterministic Finite Automaton — a first-principles approach that bridges Theory of Computation with practical network/endpoint security. This formal grounding makes the design verifiable and provably correct.
- Multi-Payload Button-Driven Architecture: Four distinct, production-realistic attack scenarios are embedded in firmware and selectable via a single GPIO button press — no re-flashing required. This makes live classroom demonstrations seamless and eliminates downtime between scenarios.
- Zero-Driver, Zero-Interaction Attack: The ESP32 firmware's USB descriptor engineering ensures the attack begins within 3 seconds of plug-in on any Windows machine without any driver installation, admin prompt, or user interaction — demonstrating the true silent nature of the BadUSB threat.
- OS-Level, Device-Agnostic Defense: The firewall's use of a kernel-level WH_KEYBOARD_LL hook makes it effective against any number of simultaneous BadUSB devices, regardless of USB port, because all HID keyboard events funnel through a single OS hook chain.
- Awareness-Focused Payload Design: All four attack scenarios are deliberately educational — they open awareness messages, launch the institution's own website, run a demo awareness script, and list Wi-Fi profiles. This makes the project ethically suitable for academic demonstration without causing harm.

10. Results

10.1 Attack Results

- The ESP32-S3 was successfully enumerated by Windows 10/11 as a standard USB HID keyboard within approximately 2–3 seconds of plug-in, with no driver installation prompt, no UAC popup, and no user interaction of any kind.
- All four attack payloads executed reliably across 10 consecutive test runs, demonstrating the repeatability and consistency of the BadUSB injection method.
- Payload 3 (PowerShell with -ExecutionPolicy Bypass) successfully bypassed Windows Script Block Logging and executed the PS1 script, demonstrating how BadUSB can circumvent default security policies.
- Payload 4 (netsh wlan show profiles) successfully enumerated all saved Wi-Fi SSIDs, highlighting the data reconnaissance capability possible with a 5-second physical attack.

10.2 Defense Results

- The keystroke firewall (firewall.py) successfully blocked 100% of all BadUSB keystroke injections when in BLOCK state across all four payload types and all 10 test runs.
- The firewall achieved sub-100ms response time from TOGGLE button press to full hook installation, making it effectively instantaneous from a user perspective.
- The DFA state toggle mechanism functioned correctly across all test iterations, with no state corruption, hook leaks, or memory issues observed during a 30-minute continuous operation test.
- The Tkinter GUI displayed accurate real-time visual feedback (ALLOW / BLOCKING) for every state transition without any UI lag or rendering failures.

Overall Project Outcome

- Attack demonstrated: ESP32-S3 successfully emulates a USB HID keyboard and executes all 4 payloads silently.
- Defense demonstrated: Python keystroke firewall blocks 100% of injected keystrokes in BLOCK state.
- Automata model validated: DFA transitions function correctly as specified by the formal definition.
- Educational value confirmed: Project successfully used in a live cybersecurity demonstration session.

11. Challenges Faced

11.1 Hardware / Firmware Challenges

- USB Enumeration Timing: The Win+R dialog takes approximately 800–1200ms to fully render, CMD takes 700–900ms, and PowerShell takes 1200–1500ms. Delays shorter than these caused keystrokes to be injected before the target window had focus, resulting in partially-executed payloads. Extensive testing on multiple PC configurations was required to find reliable timing values.
- HID Keycode Character Mapping: Building the complete send_char() function required manually mapping every required ASCII character to its USB HID keycode and modifier combination. Special characters such as ':', '%', '_', and '\\' required Shift modifier combinations. The USB HID Usage Tables specification (v1.3) was consulted extensively.

- GPIO Button Debouncing: The physical button on GPIO 0 exhibited contact bounce, causing a single press to register as 2–3 presses, prematurely advancing the payload counter. The issue was resolved by adding a 50ms polling loop that requires the GPIO to remain LOW for a full polling cycle before registering the press.
- PowerShell Execution Policy String: The payload string 'powershell -ExecutionPolicy Bypass\n' contains uppercase characters, spaces, and the '-' character, each requiring careful HID keycode mapping. An early error in the '-' mapping caused the PowerShell execution policy flag to be silently dropped.

11.2 Software / Defense Challenges

- Administrator Privilege Requirement: Python's keyboard library requires administrator privileges on Windows to install a WH_KEYBOARD_LL hook. Standard user accounts cannot install low-level hooks, requiring the firewall to be launched as Administrator. This is a usability limitation that requires documentation and user guidance.
- Full Keyboard Suppression Scope: The keyboard.hook(suppress=True) mechanism suppresses all keyboards simultaneously — including the physical keyboard. While this provides complete BadUSB neutralization, it also means the user cannot type while the firewall is active. A future improvement would use HID device-specific filtering via the Windows Raw Input API to whitelist specific physical keyboards.
- Hook Cleanup on Crash: If the Python application terminates unexpectedly, the keyboard hook is not guaranteed to be removed, potentially leaving the OS with all keyboard input suppressed until reboot. Proper exception handling and atexit cleanup were added to mitigate this risk.
- Cross-Platform Limitations: The keyboard library's suppress=True behavior is only fully supported on Windows. The firewall would need significant rework to function on Linux (evdev/X11 hooks) or macOS (CGEventTap), limiting the current implementation to Windows environments.

12. Conclusion

This project successfully demonstrates one of the most dangerous and underappreciated attack vectors in modern cybersecurity: the BadUSB attack. By programming an ESP32-S3 microcontroller to emulate a USB Human Interface Device using the TinyUSB stack, we showed that a low-cost, easily concealable piece of hardware can silently compromise any Windows computer within seconds of physical connection — no driver installation, no user interaction, and no warning signs.

The four demonstration payloads — ranging from awareness message injection to PowerShell execution policy bypass and Wi-Fi profile enumeration — illustrate the full spectrum of harm that a real-world BadUSB attacker could inflict. The fact that all of this occurs through a channel (USB HID) that most endpoint security tools ignore makes BadUSB an exceptionally potent threat.

The keystroke firewall, developed in Python and formally modelled as a Deterministic Finite Automaton, provides an effective proof-of-concept defense. By operating at the OS kernel hook level, it neutralizes all HID keyboard input — from any device — when in BLOCK state. The formal DFA model gives the design theoretical rigour: the two-state machine is provably correct, verifiable, and well-defined. This demonstrates how classical automata theory from the Theory of Computation can be directly applied to the design of practical cybersecurity systems.

The project imparts an essential lesson for cybersecurity professionals and users alike: physical access is a critical attack surface. USB ports are entry points not just for data, but for commands. Organizations should implement USB device allowlisting policies, disable USB ports on sensitive machines, deploy

HID-aware endpoint protection, and conduct regular user awareness training to mitigate the BadUSB risk.

Key Takeaways

- BadUSB attacks are silent, fast (< 5 seconds), and invisible to most antivirus solutions.
- USB HID spoofing requires no malicious files on the device — everything is injected as trusted keystrokes.
- Formal automata models (DFA) provide a rigorous, provably correct foundation for security system design.
- Physical security is as important as logical/software security in any comprehensive security posture.
- Effective defense requires operating at the OS level — application-level controls are insufficient against HID attacks.

13. References

#	Reference
[1]	Nohl, K., & Lell, J. (2014). BadUSB — On Accessories that Turn Evil. Presentation at Black Hat USA 2014. SR Labs, Berlin, Germany.
[2]	TinyUSB Project. (2024). hathach/tinyusb — Open Source USB Host/Device Stack for Embedded Systems. GitHub Repository. https://github.com/hathach/tinyusb
[3]	Espressif Systems. (2024). ESP-IDF Programming Guide — ESP32-S3 USB OTG & TinyUSB Integration. https://docs.espressif.com/projects/esp-idf
[4]	boppreh. (2024). keyboard — Python Library for Global Hotkeys and Low-level Keyboard Control. https://github.com/boppreh/keyboard
[5]	USB Implementers Forum (USB-IF). (2022). HID Usage Tables for Universal Serial Bus — Version 1.4. https://usb.org/hid
[6]	Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd Edition). Pearson Education.
[7]	Microsoft Corporation. (2024). SetWindowsHookEx function — WH_KEYBOARD_LL Low-Level Keyboard Hook. Windows API Documentation. https://learn.microsoft.com/en-us/windows/win32/api/winuser
[8]	MITRE Corporation. (2024). ATT&CK Technique T1091 — Replication Through Removable Media. MITRE ATT&CK Framework. https://attack.mitre.org/techniques/T1091/
[9]	Espressif Systems. (2023). ESP32-S3 Datasheet — Series v1.4. https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
[10]	Lameiro, F. & Belo, T. E. (2022). BadUSB in Practice: An Evaluation of USB HID Attacks Against Modern Operating Systems. SECRYPT 2022.